

COP509 Notebook 2: Extraction, Alignment And Classification

This notebook is the assessed Task 2 notebook. It demonstrates recommendation extraction, response-unit extraction, alignment, classification, confidence metadata, validation evidence and an optional interactive inspection demo.

The notebook reads the final validated outputs from project-relative paths and does not overwrite `final_recommendations_246.json`, `evaluation_predictions.csv`, or any source data.

1. Coursework Task Overview

Task 2 converts paired report and government-response PDFs into recommendation-level rows. In the current final output, this means **246 recommendation rows across the full 8-pair / 16-PDF corpus**. For each row the project records the extracted recommendation, matched response text, classification label, confidence values and validation metadata.

Full manual labels are not available for all 246 rows. This notebook therefore reports prediction-only evidence: validation checks, row counts, label distributions, confidence summaries and inspected examples. Accuracy and F1 should only be calculated if a full 246-row manual benchmark is added later.

```
In [1]: from __future__ import annotations

import json
import logging
import sys
import warnings
from collections import Counter, defaultdict
from pathlib import Path

import pandas as pd
from IPython.display import Markdown, display

pd.set_option("display.max_colwidth", 140)
warnings.filterwarnings("ignore")

def find_project_root(start: Path | None = None) -> Path:
    """Find the project root from the repository layout, not a machine-specific
    start = (start or Path.cwd()).resolve()
    required_dirs = ("src", "data", "notebooks", "outputs")
    coursework_names = ("COP509_Coursework.pdf", "COP509_coursework.pdf")
    for candidate in [start, *start.parents]:
        has_required_dirs = all((candidate / name).is_dir() for name in required
        has_coursework_file = any((candidate / name).exists() for name in course
        if has_required_dirs and (has_coursework_file or (candidate / "outputs"
        return candidate
```

```

raise FileNotFoundError(
    "Could not find the project root. Expected directories: src/, data/, not
    "and either the final export or the COP509 coursework PDF."
)

PROJECT_ROOT = find_project_root()
DATA_DIR = PROJECT_ROOT / "data"
DATA_RAW_DIR = DATA_DIR / "raw"
OUTPUTS_DIR = PROJECT_ROOT / "outputs"
FINAL_EXPORT_PATH = OUTPUTS_DIR / "final_recommendations_246.json"
EVALUATION_PREDICTIONS_PATH = OUTPUTS_DIR / "evaluation_predictions.csv"
QA_MATRIX_PATH = DATA_DIR / "ground_truth" / "qa_matrix_queries.json"

# Backward-compatible alias used by older display cells.
RAW_DIR = DATA_RAW_DIR
if str(PROJECT_ROOT) not in sys.path:
    sys.path.insert(0, str(PROJECT_ROOT))

logging.getLogger("src.preprocessing").setLevel(logging.ERROR)
logging.getLogger("src.pdf_loader").setLevel(logging.ERROR)
print(f"Project root detected: {PROJECT_ROOT.name}")
print(f"Raw PDF directory: {DATA_RAW_DIR.relative_to(PROJECT_ROOT)}")
print(f"Final export: {FINAL_EXPORT_PATH.relative_to(PROJECT_ROOT)}")

```

Project root detected: cop509-coursework

Raw PDF directory: data\raw

Final export: outputs\final_recommendations_246.json

2. Dataset And Pair Setup

The shared preset map is the source of truth for the full available corpus. It identifies **8 document pairs / 16 PDFs**: coursework-given pairs plus extension pairs used to test the extraction, alignment and classification workflow.

```

In [2]: from backend.core.presets import PRESETS, validate_preset_files

preset_rows = []
for preset in PRESETS.values():
    try:
        validate_preset_files(preset)
        files_ok, notes = True, ""
    except FileNotFoundError as exc:
        files_ok, notes = False, str(exc)
    special = []
    if preset.allow_response_heading_recommendation_fallback:
        special.append("response-heading fallback")
    if preset.inline_recommendation_chapter_prefix:
        special.append(f"inline prefix {preset.inline_recommendation_chapter_pre")
    if preset.select_committee_conclusions_section:
        special.append("select-committee section")
    preset_rows.append({
        "preset_id": preset.id,
        "label": preset.label,
        "dataset_group": preset.dataset_group,
        "policy_pdf": preset.policy_pdf.name,
        "response_pdf": preset.response_pdf.name,
        "files_ok": files_ok,

```

```

        "special_extraction": ", ".join(special),
        "notes": notes,
    })

preset_df = pd.DataFrame(preset_rows)
if not preset_df["files_ok"].all():
    display(preset_df)
    raise FileNotFoundError("One or more preset PDFs are missing.")
all_preset_pdf_names = sorted({path.name for preset in PRESETS.values() for path
print(f"Full available preset corpus: {len(PRESETS)} document pairs / {len(all_p
print("Task 2 final output is reported across the full 8-pair corpus below.")
display(preset_df)

```

Full available preset corpus: 8 document pairs / 16 PDFs

Task 2 final output is reported across the full 8-pair corpus below.

	preset_id	label	dataset_group	policy_pdf	
0	behaviour_change	Behaviour Change	coursework_given	Behaviour-Change-Report- Recomm.pdf	Behav
1	post_office	Post Office Horizon Inquiry	coursework_given	PostOfficeHorizon-I- Inquiry- Recomm.pdf	Po
2	space_economy	The Space Economy	coursework_given	TheSpaceEconomyReport.pdf	TheSi
3	covid_inquiry	UK Covid-19 Inquiry Module 1	coursework_given	UK-Covid-19-Inquiry- Module-1-Recomm.pdf	19_Inqui
4	blood_inquiry	Infected Blood Inquiry	coursework_given	Volume_1-Blood-Inquiry- Recomm.pdf	
5	grenfell_phase2	Grenfell Tower Inquiry — Phase 2	extra_found	Grenfell-Phase2-Volume7- Recomm.pdf	Gre
6	covid_inquiry_module2	UK Covid-19 Inquiry Module 2	extra_found	UK-Covid-19-Inquiry- Module-2-Recomm.pdf	19_Inqui
7	summer_2024_disorder	Police response to the 2024 summer disorder	extra_found	Summer2024-Disorder- Recomm.pdf	



3. Source-Code Mirroring

The notebook imports the same shared project modules used by the final validator. Notebook-local code is limited to display helpers and summary tables.

```
In [3]: from src.pdf_loader import extract_pages
from src.extraction import extract_recommendations
from src.response_units import extract_response_units
from src.alignment import match_recommendations_to_response_units
from src.classification import classify_response, classify_with_confidence, norm
from scripts.validate_recommendation_export import (
    classification_failures,
    metadata_failures,
    recommendation_cleanup_failures,
    response_leakage_failures,
)

TASK2_LABELS = ["accepted", "partial", "rejected", "not_addressed"]
LABEL_DISPLAY = {
    "accepted": "accepted",
    "partial": "partially accepted",
    "partially_accepted": "partially accepted",
    "rejected": "rejected",
    "not_addressed": "not addressed",
}
print("Loaded source modules for loading, extraction, response units, alignment
```

Loaded source modules for loading, extraction, response units, alignment and classification.

4. Final Validated Export Baseline

The selected evidence baseline is the canonical project-relative file `outputs/final_recommendations_246.json`. The cell below reads that file, checks the expected row counts and leaves the JSON unchanged.

```
In [4]: EXPECTED_PAIR_COUNTS = {
    "behaviour_change": 33,
    "post_office": 19,
    "space_economy": 40,
    "covid_inquiry": 10,
    "blood_inquiry": 58,
    "grenfell_phase2": 58,
    "covid_inquiry_module2": 19,
    "summer_2024_disorder": 9,
}

EXPECTED_CLASS_COUNTS = {"accepted": 104, "partial": 139, "rejected": 1, "not_ad
FINAL_EXPORT_CANDIDATES = [FINAL_EXPORT_PATH]

def load_export(path: Path) -> list[dict]:
    payload = json.loads(path.read_text(encoding="utf-8"))
    rows = payload.get("recommendations", payload) if isinstance(payload, dict)
    if not isinstance(rows, list):
        raise ValueError(f"{path} does not contain a recommendation list")
    return rows

selected_export_path, final_rows = None, None
for candidate in FINAL_EXPORT_CANDIDATES:
```

```

if not candidate.exists():
    continue
rows = load_export(candidate)
if (
    len(rows) == 246
    and dict(Counter(str(r.get("document_pair")) for r in rows)) == EXPECTED
    and dict(Counter(str(r.get("classification")) for r in rows)) == EXPECTED
):
    selected_export_path, final_rows = candidate, rows
    break

if final_rows is None:
    found = [str(path) for path in FINAL_EXPORT_CANDIDATES if path.exists()]
    raise FileNotFoundError("No validated 246-row final export found. Expected:

final_df = pd.json_normalize(final_rows, sep=".")
print("Selected final export evidence baseline:")
print(selected_export_path.relative_to(PROJECT_ROOT))
print("Canonical evidence name used in this notebook: final_recommendations_246.")
print(f"Rows loaded: {len(final_df)}")

```

Selected final export evidence baseline:

outputs\final_recommendations_246.json

Canonical evidence name used in this notebook: final_recommendations_246.json

Rows loaded: 246

5. Final Full Export Summary

The compact export label `partial` means partially accepted.

```

In [5]: pair_summary = pd.DataFrame({
    "document_pair": list(EXPECTED_PAIR_COUNTS),
    "expected_rows": list(EXPECTED_PAIR_COUNTS.values()),
    "actual_rows": [int((final_df["document_pair"] == pair).sum()) for pair in E
    })
pair_summary["matches_expected"] = pair_summary["expected_rows"] == pair_summary

class_counts = final_df["classification"].value_counts().to_dict()
class_summary = pd.DataFrame({
    "export_label": TASK2_LABELS,
    "display_label": [LABEL_DISPLAY[label] for label in TASK2_LABELS],
    "expected_rows": [EXPECTED_CLASS_COUNTS[label] for label in TASK2_LABELS],
    "actual_rows": [int(class_counts.get(label, 0)) for label in TASK2_LABELS],
    })
class_summary["percentage"] = (class_summary["actual_rows"] / len(final_df) * 100)
class_summary["matches_expected"] = class_summary["expected_rows"] == class_summary["actual_rows"]

print(f"Final row count: {len(final_df)}")
display(pair_summary)
display(class_summary)

```

Final row count: 246

	document_pair	expected_rows	actual_rows	matches_expected
0	behaviour_change	33	33	True
1	post_office	19	19	True
2	space_economy	40	40	True
3	covid_inquiry	10	10	True
4	blood_inquiry	58	58	True
5	grenfell_phase2	58	58	True
6	covid_inquiry_module2	19	19	True
7	summer_2024_disorder	9	9	True

	export_label	display_label	expected_rows	actual_rows	percentage	matches_expected
0	accepted	accepted	104	104	42.3	True
1	partial	partially accepted	139	139	56.5	True
2	rejected	rejected	1	1	0.4	True
3	not_addressed	not addressed	2	2	0.8	True

6. Confidence, Metadata And Validation Checks

The final export includes overall confidence plus debug fields for alignment confidence, lexical similarity, classification confidence, classifier method, alignment method and confidence factors.

```
In [6]: confidence_summary = pd.DataFrame([
    {"metric": "overall confidence", "mean": final_df["confidence"].mean(), "min":
    {"metric": "alignment confidence", "mean": final_df["debug.alignment_confidence"].mean(), "min":
    {"metric": "classification confidence", "mean": final_df["debug.classification_confidence"].mean(), "min":
]).round(3)
method_summary = final_df["debug.alignment_method"].value_counts(dropna=False).sort_index()
classifier_summary = final_df["debug.classifier_method"].value_counts(dropna=False).sort_index()
display(confidence_summary)
display(method_summary)
display(classifier_summary)

def duplicate_id_failures(rows: list[dict]) -> dict[str, list[str]]:
    by_pair = defaultdict(list)
    for row in rows:
        by_pair[str(row.get("document_pair"))].append(str(row.get("id")))
    return {pair: sorted([item for item, count in Counter(ids).items() if count > 1]) for pair in by_pair}

def blank_id_failures(rows: list[dict]) -> list[tuple[str, str]]:
    return [(str(r.get("document_pair")), str(r.get("id"))) for r in rows if r.get("id") == ""]
```

```

validation_df = pd.DataFrame([
    {"check": "row count", "status": "ok" if len(final_rows) == 246 else "failed"},
    {"check": "document-pair counts", "status": "ok" if pair_summary["matches_ex"]},
    {"check": "classification distribution", "status": "ok" if class_summary["ma"]},
    {"check": "duplicate IDs", "status": "ok" if not duplicate_id_failures(final_rows)},
    {"check": "blank IDs", "status": "ok" if not blank_id_failures(final_rows) e},
    {"check": "response leakage", "status": "ok" if not response_leakage_failure},
    {"check": "recommendation cleanup", "status": "ok" if not recommendation_cle},
    {"check": "metadata completeness", "status": "ok" if not metadata_failures(f},
    {"check": "classification corrections", "status": "ok" if not classification
])
display(validation_df)
if not (validation_df["status"] == "ok").all():
    raise AssertionError("One or more final export validation checks failed.")

```

	metric	mean	min	max
0	overall confidence	0.935	0.8	0.95
1	alignment confidence	0.938	0.9	0.95
2	classification confidence	0.787	0.6	0.92

	alignment_method	rows
0	exact_label	148
1	structured_grouped	52
2	structure	30
3	structured_ordinal	10
4	structured_section	6

	classifier_method	rows
0	rule_based	246

	check	status	details
0	row count	ok	246
1	document-pair counts	ok	see pair summary
2	classification distribution	ok	see class summary
3	duplicate IDs	ok	none
4	blank IDs	ok	none
5	response leakage	ok	none
6	recommendation cleanup	ok	none
7	metadata completeness	ok	none
8	classification corrections	ok	none

7. Live Pipeline Demonstration On One Pair

The final export above is loaded read-only. This section runs the shared project modules on one representative pair so the extraction, response-unit and alignment stages are visible without changing final outputs.

```
In [7]: DEMO_PRESET_ID = "behaviour_change"
demo_preset = PRESETS[DEMO_PRESET_ID]
policy_pages = extract_pages(demo_preset.policy_pdf)
response_pages = extract_pages(demo_preset.response_pdf)
demo_recommendations = extract_recommendations(
    policy_pages,
    response_pages_fallback=(response_pages if demo_preset.allow_response_headin
    inline_chapter_prefix=demo_preset.inline_recommendation_chapter_prefix,
    select_committee_section=demo_preset.select_committee_conclusions_section,
)
demo_response_units = extract_response_units(response_pages)
demo_alignments = match_recommendations_to_response_units(demo_recommendations,
print(f"Demo preset: {DEMO_PRESET_ID} - {demo_preset.label}")
print(f"Policy pages: {len(policy_pages)}")
print(f"Response pages: {len(response_pages)}")
print(f"Recommendations extracted: {len(demo_recommendations)}")
print(f"Response units extracted: {len(demo_response_units)}")
print(f"Alignment candidates returned: {len(demo_alignments)}")
```

Demo preset: behaviour_change - Behaviour Change

Policy pages: 109

Response pages: 18

Recommendations extracted: 33

Response units extracted: 22

Alignment candidates returned: 99

8. Recommendation Extraction And Response Matching

Recommendation extraction records method and confidence. Response-unit matching returns alignment method, similarity and alignment confidence.

```
In [8]: demo_rec_df = pd.DataFrame(demo_recommendations)
display(demo_rec_df["extraction_method"].value_counts().rename_axis("extraction_
display(demo_rec_df[["rec_id", "item_label", "page_number", "extraction_method",

unit_df = pd.DataFrame(demo_response_units)
if not unit_df.empty:
    unit_df = unit_df.assign(response_preview=unit_df["response_text"].fillna("")
    display(unit_df[["recommendation_labels", "page_start", "page_end", "extract

demo_align_df = pd.DataFrame(demo_alignments)
if not demo_align_df.empty:
    display(demo_align_df.groupby("match_method", as_index=False).agg(rows=("rec
    display(demo_align_df[["rec_id", "match_method", "similarity", "alignment_co
```

	extraction_method	rows
0	structured section	33

rec_id	item_label	page_number	extraction_method	confidence	text	
0	0	8.1	67	structured section	0.86	The idea of the Government intervening to change people's behaviour will often be controversial, and so it is important that ministers a...
1	1	8.2	67	structured section	0.86	There is a lack of applied research at a population level to support specific interventions to change the behaviour of large groups of p...
2	2	8.3	67	structured section	0.86	We acknowledge that there will be occasions when it is legitimate for a government not to implement behaviour change interventions for w...
3	3	8.4	67	structured section	0.92	We agree with the principle, stated in the Government's Principles of Scientific Advice , that ministers should explain publicly their r...
4	4	8.5	67	structured section	0.92	We urge ministers to consult their departmental Chief Scientific Advisers about whether the amount of money spent on applied behaviour c...
5	5	8.6	68	structured section	0.92	We recommend that, at the earliest opportunity, the Government appoint a Chief Social Scientist who reports to the Government Chief Scie...

rec_id	item_label	page_number	extraction_method	confidence	text	
6	6	8.7	68	structured section	0.92	We further recommend that the Government consider whether existing mechanisms for the provision of social scientific advice, in particul...
7	7	8.8	68	structured section	0.92	Departmental Chief Scientific Advisers, whether or not they have experience of the sciences of human behaviour, should be responsible fo...

	recommendation_labels	page_start	page_end	extraction_confidence	boundary_reason
0	[8.1]	3	3	0.95	next_heading
1	[8.2]	3	3	0.95	next_heading
2	[8.3, 8.4]	4	4	0.75	multi_label_block
3	[8.5]	4	4	0.95	next_heading
4	[8.6, 8.7]	4	5	0.75	multi_label_block
5	[8.8, 8.9]	5	6	0.75	multi_label_block

	recommendation_labels	page_start	page_end	extraction_confidence	boundary_reason
6	[8.10, 8.11]	6	6	0.75	multi_label_block

7	[8.12, 8.13]	6	7	0.75	multi_label_block
---	--------------	---	---	------	-------------------

	match_method	rows	mean_similarity	mean_alignment_confidence
0	exact_label	33	0.950	0.950
1	semantic	66	0.127	0.127

rec_id	match_method	similarity	alignment_confidence	page_number	matched_text	
0	0	exact_label	0.950000	0.950000	3	The Government agrees with the Committee's conclusion. It is important that the behaviour of the intervention draw from the available evidence base.
1	0	semantic	0.128381	0.128381	11	The Government agrees with the Committee's recommendation. However, we would emphasise that Change4Life presents a number of challenges.
2	0	semantic	0.126074	0.126074	10	The Government agrees that it is important for local authorities to have the ability to tailor their interventions to local populations..
3	1	exact_label	0.950000	0.950000	3	The Government agrees with the Committee's conclusion. As with all policy decisions, we believe that it is important that the Government.
4	1	semantic	0.130277	0.130277	8	As the Committee has acknowledged collaboration between the Government, public health, commercial and voluntary organisation presents a u.
5	1	semantic	0.123090	0.123090	18	The Department for Transport drew local

rec_id	match_method	similarity	alignment_confidence	page_number	matched_text	
					authorities attention to wide range of research and evaluation evidence where the Local Sustai.	
6	2	exact_label	0.950000	0.950000	4	The Governmen agrees with th Committee' conclusion. A with all polic decisions and i line with th GCSA Guideline on the use of s.
7	2	semantic	0.131071	0.131071	11	The Governmen agrees with th Committee' recommendation However, w would emphasiz that Change4Lif presents number of challenges

9. Classification Labels

The classifier returns `accepted`, `partially_accepted`, `rejected` or `not_addressed`. The final export normalises `partially_accepted` to the compact `partial` value.

```
In [9]: alignments_by_rec = defaultdict(list)
for alignment in demo_alignments:
    alignments_by_rec[int(alignment["rec_id"])].append(dict(alignment))

classified_demo_rows = []
for rec in demo_recommendations:
    rec_matches = sorted(alignments_by_rec.get(int(rec["rec_id"]), []), key=lambda x: x["similarity"])
    best = rec_matches[0] if rec_matches else None
    if best:
        raw_label, class_conf = classify_with_confidence(str(best.get("matched_text")))
        label = normalize_label(raw_label)
    else:
        label, class_conf = "not_addressed", 0.0
    export_label = "partial" if label == "partially_accepted" else label
    classified_demo_rows.append({
        "item_label": rec.get("item_label"),
        "classification": LABEL_DISPLAY.get(export_label, export_label),
        "classification_confidence": round(float(class_conf), 3),
        "alignment_confidence": round(float((best or {}).get("alignment_confidence")), 3)
    })
```

```
        "match_method": (best or {}).get("match_method"),
        "recommendation_preview": str(rec.get("text", ""))[:120] + "...",
        "response_preview": str((best or {}).get("matched_text", ""))[:120] + ".
    })
classified_demo_df = pd.DataFrame(classified_demo_rows)
display(classified_demo_df.head(10))
display(classified_demo_df["classification"].value_counts().rename_axis("classif
```

	item_label	classification	classification_confidence	alignment_confidence	match_metho
0	8.1	accepted	0.78	0.95	exact_lab
1	8.2	accepted	0.85	0.95	exact_lab
2	8.3	accepted	0.75	0.95	exact_lab
3	8.4	accepted	0.75	0.95	exact_lab
4	8.5	accepted	0.85	0.95	exact_lab
5	8.6	partially accepted	0.70	0.95	exact_lab
6	8.7	partially accepted	0.70	0.95	exact_lab

	item_label	classification	classification_confidence	alignment_confidence	match_metho
7	8.8	accepted	0.92	0.95	exact_label
8	8.9	accepted	0.92	0.95	exact_label
9	8.10	accepted	0.85	0.95	exact_label

	classification	rows
0	partially accepted	19
1	accepted	14

10. Prediction-Only Evaluation Note

A full manual ground-truth label set is not available for all 246 final rows. For that reason, this notebook does not report overall accuracy, precision, recall or F1 for Task 2. Instead, it reports prediction-only evidence: final row counts, label distributions, validation checks, confidence summaries and inspected examples.

If a complete 246-row manual benchmark is added later, accuracy and F1 can be calculated against that benchmark. The existing `labels.json` file is not a full-row benchmark and is not used for that purpose.

11. Examples And Hard Cases

These static tables keep representative evidence visible in PDF export.

```

In [10]: example_keys = {
    ("behaviour_change", "8.1"): "accepted example",
    ("post_office", "4"): "partially accepted example",
    ("post_office", "13"): "rejected example",
    ("covid_inquiry_module2", "1"): "not addressed example",
    ("covid_inquiry_module2", "13"): "not addressed example",
}

example_df = final_df[final_df.apply(lambda row: (str(row["document_pair"]), str(row["evidence_note"])), axis=1, result_type="expand").isin(example_keys)]
example_df["evidence_note"] = example_df.apply(lambda row: example_keys[(str(row["document_pair"]), str(row["evidence_note"]))], axis=1, result_type="expand").iloc[0]
example_df["recommendation_preview"] = example_df["recommendation_text"].fillna("")
example_df["response_preview"] = example_df["matched_response_text"].fillna("")
display(example_df[["evidence_note", "document_pair", "id", "classification", "confidence"]])

hard_notes = {
    ("behaviour_change", "8.33"): "response heading repeats 8.32, so exact number is 8.32",
    ("space_economy", "30"): "trailing section-heading cleanup prevents recommendation extraction",
    ("blood_inquiry", "4a i"): "nested labels are preserved for grouped response",
    ("grenfell_phase2", "58"): "inline/fallback extraction handles embedded inquiry",
    ("summer_2024_disorder", "1"): "select-committee extraction excludes conclusions"
}

hard_df = final_df[final_df.apply(lambda row: (str(row["document_pair"]), str(row["case_note"])), axis=1, result_type="expand").isin(hard_notes)]
hard_df["case_note"] = hard_df.apply(lambda row: hard_notes[(str(row["document_pair"]), str(row["case_note"]))], axis=1, result_type="expand").iloc[0]
hard_df["recommendation_preview"] = hard_df["recommendation_text"].fillna("")
hard_df["response_preview"] = hard_df["matched_response_text"].fillna("").str.strip()
display(hard_df[["case_note", "document_pair", "id", "classification", "confidence"]])

```

	evidence_note	document_pair	id	classification	confidence	debug.alignment
0	accepted example	behaviour_change	8.1	accepted	0.95	
218	not addressed example	covid_inquiry_module2	1	not_addressed	0.85	
230	not addressed example	covid_inquiry_module2	13	not_addressed	0.85	
45	rejected example	post_office	13	rejected	0.95	
36	partially accepted example	post_office	4	partial	0.95	



	case_note	document_pair	id	classification	confidence	debug.align
32	response heading repeats 8.32, so exact numbering is insufficient	behaviour_change	8.33	partial	0.95	
109	nested labels are preserved for grouped response structure	blood_inquiry	4a i	accepted	0.90	struct
217	inline/fallback extraction handles embedded inquiry recommendations	grenfell_phase2	58	accepted	0.95	
66	trailing section-heading cleanup prevents recommendation leakage	space_economy	30	partial	0.95	
237	select-committee extraction excludes conclusion-only items	summer_2024_disorder	1	partial	0.95	

12. Optional Interactive Extraction And Alignment Demo

The static evidence above is sufficient for PDF marking: it includes the final 246-row summary, pair counts, classification distribution, validation checks, representative examples and hard cases. The widget below is an optional live notebook demo for inspecting one pair interactively.

```
In [11]: from src.widgets.extraction_explorer import show as show_extraction_explorer

LABEL_COLOURS = {
    "accepted": "#198754",
    "partial": "#fd7e14",
    "partially_accepted": "#fd7e14",
    "rejected": "#dc3545",
    "not_addressed": "#6c757d",
}

show_extraction_explorer(demo_recommendations, demo_alignments, classify_respons
VBox(children=(HBox(children=(Dropdown(description='Document:', layout=Layout(wid
th='40%')), options=('All load...
```

Optional Final-Row Inspector

This small inspector is separate from the widget above. It lets a live notebook user choose any final export row and view the recommendation, matched response, classification, confidence and provenance. The static examples above remain the PDF evidence.

```
In [12]: import html
import ipywidgets as widgets
from IPython.display import HTML, display

final_export_df = final_df.reset_index(drop=True).copy()

def _row_label(row: pd.Series) -> str:
    return f"{row['document_pair']} {row['id']} - {LABEL_DISPLAY.get(row['classi

def _render_final_row(row_index: int) -> HTML:
    row = final_export_df.iloc[int(row_index)]
    recommendation = html.escape(str(row.get('recommendation_text', '') or ''))
    response = html.escape(str(row.get('matched_response_text', '') or ''))
    classification = html.escape(str(LABEL_DISPLAY.get(row.get('classification')
    confidence = html.escape(str(row.get('confidence', '')))
    align_method = html.escape(str(row.get('debug.alignment_method', '')))
    align_conf = html.escape(str(row.get('debug.alignment_confidence', '')))
    response_page = html.escape(str(row.get('matched_response_page', '')))
    row_id = html.escape(f"{row.get('document_pair')} {row.get('id')}")
    return HTML(f"""
<div style="border:1px solid #ddd;padding:10px;border-radius:6px;font-family
    <div><strong>Recommendation ID:</strong> {row_id}</div>
    <div><strong>Classification:</strong> {classification}</div>
    <div><strong>Confidence:</strong> {confidence}</div>
    <div><strong>Alignment method:</strong> {align_method} (confidence {align_
    <div><strong>Matched response page:</strong> {response_page}</div>
    <hr style="border:none;border-top:1px solid #eee;margin:8px 0;"/>
    <div><strong>Recommendation text</strong></div>
```

```

        <div style="max-height:180px;overflow:auto;white-space:pre-wrap;">{recomme
        <div style="margin-top:8px;"><strong>Matched response text</strong></div>
        <div style="max-height:220px;overflow:auto;white-space:pre-wrap;">{respons
    </div>
    """)

row_dropdown = widgets.Dropdown(
    options=[(_row_label(row), idx) for idx, row in final_export_df.iterrows()],
    value=0,
    description="Row",
    layout=widgets.Layout(width="100%"),
)
row_output = widgets.Output()

def _update_row(change=None) -> None:
    with row_output:
        row_output.clear_output(wait=True)
        display(_render_final_row(row_dropdown.value))

row_dropdown.observe(_update_row, names="value")
_update_row()
display(widgets.VBox([row_dropdown, row_output]))

```

VBox(children=(Dropdown(description='Row', layout=Layout(width='100%'), options=((('behaviour_change 8.1 - acce...

13. Challenges, Optional Web Interface And Limitations

Challenges and resolutions. The workflow handles unreliable numbering, varied recommendation layouts, quoted recommendation text inside responses, nested labels and incomplete manual ground truth. It uses documented preset-specific settings where needed.

Optional web interface. The FastAPI/Vite interface exposes the same project workflow interactively, but this notebook does not depend on a deployed web interface or screenshots.

Limitations and future work. Add full 246-row manual labels for accuracy/F1, keep expanding hard-case tests for new document layouts, improve OCR quality auditing and consider a second-stage semantic reranker for weak matches after submission.

14. Final Conclusion

The final Task 2 evidence shows 246 recommendation-level rows across the full corpus of eight document pairs / 16 PDFs, with the expected pair counts and classification distribution. The notebook uses the shared project modules, preserves confidence metadata, provides static and optional interactive evidence, and avoids overwriting final outputs.